

B.Tech Term Project Task Allocation

1. Project Background

Anaemia is a major public-health problem in India, especially in rural areas, where invasive blood testing is costly and slow. This project contributes to a research programme developing a smartphone-based, non-invasive anaemia screening system that analyses the colour of the eye conjunctiva. A central challenge is that images are captured on many different phones under varying lighting, which affects accuracy.

Two coding work packages are defined below. Each is a self-contained software deliverable with clear specifications. Students are expected to implement to the given specification; research design and methodology are provided by the supervisor.

2. Work Package A — Data Pipeline and Benchmarking

Objective

Build reusable Python code to load, organise, and benchmark publicly available anaemia image datasets, so that experiments can be run consistently across datasets.

Tasks

- **Dataset loader:** implement a metadata builder for the CP-AnemiC (Ghana) dataset — parse the images and labels into a single, clean CSV file (patient ID, image path, haemoglobin value, gender, age, label).
- **ROI preparation:** adapt the existing region-of-interest extraction pipeline to run on CP-AnemiC and, if time permits, on public nail and palm datasets.
- **Baseline model harness:** implement a configurable training and evaluation script (transfer-learning CNN such as ResNet-50 / EfficientNet) with patient-level stratified k-fold cross-validation.
- **Metrics and figures:** produce a standard results module — accuracy, sensitivity, specificity, F1, F2, AUC — plus confusion-matrix, ROC and precision–recall plots at publication quality.
- **Documentation:** a short README explaining how to run each script and reproduce the baseline results.

Deliverables

- Clean, documented Python code (dataset loader, ROI pipeline, training/evaluation harness).
- A baseline results table for each public dataset.
- A short reproducibility guide (README).

Tools and starting point

Python, PyTorch, OpenCV, scikit-learn, pandas, matplotlib.

Acceptance criteria

- Scripts run end-to-end from the command line with a single configuration file.
- Dataset class counts and baseline metrics are reproducible and documented.
- Code is clean, commented, and organised in the agreed folder structure.

3. Work Package B — Image Capture Application

Objective

Build a simple, reliable application that guides a user to photograph the eye conjunctiva and automatically selects the best-quality frame, producing consistently named, structured images for research data collection.

Tasks

- **Guided capture:** display an on-screen alignment guide (oval) and instruct the user to bring the camera close and hold steady for about five seconds.
- **Burst capture:** capture multiple frames during the hold and store them temporarily for scoring.
- **Best-frame selection:** automatically score each frame for sharpness (focus), exposure (brightness), and glare, and keep the single best frame.
- **Structured saving:** save the selected image with a standardised filename encoding device, distance, and lighting condition, into an organised folder structure.
- **Review screen** allow the user to keep or discard the captured image.

Deliverables

- A working capture application (mobile or desktop, as agreed).
- A small pilot dataset captured with multiple devices to demonstrate the tool.
- A short user guide and setup instructions.

Tools and starting point

Any suitable stack (e.g., Python with OpenCV for a desktop prototype, or a mobile framework). The supervisor will provide the required frame-quality criteria and the file-naming convention.

Acceptance criteria

- The app captures a burst, selects the sharpest, well-exposed, glare-free frame, and saves it with the correct filename.
- Captured images are organised and consistently named.
- The tool runs reliably on at least two different devices.

